

Data Preprocessing

Data Preprocessing

What we cover

- Data Preprocessing
- Missing data handling
- Categorical data handling
- Class Imbalance
- Performance evaluation metrics
- Feature Scaling

Data Preprocessing

Data Preprocessing

- The quality and information contained within a dataset play a crucial role in the effectiveness of a machine learning algorithm.
- To ensure optimal learning, it is essential to carefully examine and preprocess the dataset before feeding it into the algorithm.
- The fundamental techniques of data preprocessing such as **handling missing data**, **scaling data**, and **feature selection** are vital for constructing robust and accurate machine learning models.

Data Preprocessing

Missing data

- In real-world applications, it is quite common for our samples to have missing values for various reasons.
 - **Data entry errors:** Missing values can occur due to human errors during data entry or data collection processes.
 - **Non-response:** In surveys or questionnaires, respondents may choose not to answer certain questions, resulting in missing values.
 - **Data corruption:** Technical issues or data transfer problems can lead to missing values in datasets.
 - **Incomplete data:** In some cases, data may be intentionally collected or recorded incompletely, leading to missing values.
 - **Data not available:** Certain information may not be available or accessible for various reasons, resulting in missing values.
 - **Privacy concerns:** Missing values may occur when certain sensitive information is intentionally omitted or masked to protect privacy.

Missing data handling

- It's important to understand the reasons behind missing values as it can influence the appropriate approach for handling them during data preprocessing.
- Dealing with missing data is a common challenge in data preprocessing. When a dataset contains missing values, it can adversely affect the performance of machine learning algorithms.
- There are several approaches to handle missing data:
 - **Sample or feature elimination:** One option is to remove instances or features that have missing values. This approach is suitable when the amount of missing data is small and removing it does not significantly impact the overall dataset.
 - **Missing values imputation:** Another approach is to fill in the missing values with estimated or imputed values. Common imputation techniques include filling missing values with the mean, median, or mode of the feature, or using more sophisticated methods like regression or k-nearest neighbors (KNN) to estimate the missing values.

Missing data handling

- There are several approaches to handle missing data:
 - **Indicator variables:** In some cases, it may be informative to create an additional binary indicator variable that represents whether a value is missing or not. This allows the algorithm to capture the potential information encoded in the missingness itself.
 - **Advanced techniques:** For more complex scenarios, advanced techniques such as matrix factorization, probabilistic modeling, or deep learning-based methods can be used to handle missing data
- It is important to carefully consider the nature of the missing data and the specific requirements of the problem at hand when choosing an appropriate method for dealing with missing data.

Data Preprocessing

Missing data handling

- Example of elimination:
- This code uses the `dropna()` function in pandas to remove rows with any missing values from the DataFrame.

```
@author: Mehdi Yousefi
"""
import pandas as pd
import csv

# Sample data with missing values
data = [
    ["Name", "Age", "Gender"],
    ["John", 25, "Male"],
    ["Alice", 32, None],
    ["Bob", None, "Male"],
    ["Emily", None, None],
]

# Write data to CSV file
with open('data.csv', mode='w', newline='') as file:
    writer = csv.writer(file)
    writer.writerows(data)

print("data.csv created successfully.")

# Load the dataset
data = pd.read_csv('data.csv')

# Print data before removing missing values
print("Data before removing missing values:")
print(data)

# Remove instances with missing values
data_without_missing = data.dropna()

# Print data after removing missing values
print("\nData after removing missing values:")
print(data_without_missing)
```

Output:

```
Data before removing missing values:
   Name  Age Gender
0  John  25.0   Male
1  Alice 32.0    NaN
2   Bob  NaN    Male
3  Emily  NaN    NaN

Data after removing missing values:
   Name  Age Gender
0  John  25.0   Male

In [4]:
```

Missing data handling

- Removing missing data may seem convenient as an approach to handling missing values. However, it has certain drawbacks:
 - By removing samples or feature columns with missing data, we risk losing valuable information and reducing the reliability of our analysis.
 - If we remove excessive feature columns, we run the risk of losing valuable information that is crucial for our classifier to distinguish between different classes.
- Interpolation techniques are widely employed as one of the most popular alternatives for handling missing values.

Data Preprocessing

Missing data handling

- Example of imputation:

```
@author: Mehdi
"""
import pandas as pd
import numpy as np

# Create a DataFrame with missing values
data = pd.DataFrame({
    'Column1': [1, 2, np.nan, 4, 5],
    'Column2': [6, np.nan, 8, 9, 10],
    'Column3': [11, 12, np.nan, 14, 15]
})

# Save the DataFrame to a CSV file
data.to_csv('data.csv', index=False)

# Load the dataset
data = pd.read_csv('data.csv')

# Print the dataset before handling missing values
print("Before handling missing values:")
print(data)

# Impute missing values with mean
data['Column1'].fillna(data['Column1'].mean(), inplace=True)
data['Column2'].fillna(data['Column2'].mean(), inplace=True)
data['Column3'].fillna(data['Column3'].mean(), inplace=True)

# Print the dataset after handling missing values
print("\nAfter handling missing values:")
print(data)
```

Output:

```
Before handling missing values:
   Column1  Column2  Column3
0      1.0      6.0     11.0
1      2.0      NaN     12.0
2      NaN      8.0      NaN
3      4.0      9.0     14.0
4      5.0     10.0     15.0

After handling missing values:
   Column1  Column2  Column3
0      1.0      6.00     11.0
1      2.0      8.25     12.0
2      3.0      8.00     13.0
3      4.0      9.00     14.0
4      5.0     10.00     15.0
```

Data Preprocessing

Missing data handling

- Example of imputation using **sklearn**:

```
Created on Sat Jun 17 21:56:37 2023

@author: Mehdi
"""

import pandas as pd
from sklearn.impute import SimpleImputer

# Create a DataFrame with missing values
data = pd.DataFrame({
    'Column1': [1, 2, None, 4, 5],
    'Column2': [6, None, 8, None, 10],
    'Column3': [None, 12, None, 14, 15]
})

# Save the DataFrame to a CSV file
data.to_csv('data.csv', index=False)

# Load the dataset
data = pd.read_csv('data.csv')

# Print the dataset before handling missing values
print("Before handling missing values:")
print(data)

# Create SimpleImputer object
imputer = SimpleImputer(strategy='mean')

# Impute missing values
data_imputed = pd.DataFrame(imputer.fit_transform(data), columns=data.columns)

# Print the dataset after handling missing values
print("\nAfter handling missing values:")
print(data_imputed)
```

Output:

```
Before handling missing values:
  Column1  Column2  Column3
0      1.0      6.0      NaN
1      2.0      NaN     12.0
2      NaN      8.0      NaN
3      4.0      NaN     14.0
4      5.0     10.0     15.0

After handling missing values:
  Column1  Column2  Column3
0      1.0      6.0  13.666667
1      2.0      8.0  12.000000
2      3.0      8.0  13.666667
3      4.0      8.0  14.000000
4      5.0     10.0  15.000000
```

Handling categorical data

- It is not uncommon to encounter real-world datasets that include one or more columns with categorical features.
- Categorical data can be further divided into two types: **nominal** and **ordinal**
- Ordinal features refer to categorical values that can be arranged or ordered in a particular sequence.
 - For instance, T-shirt size can be considered an ordinal feature because we can establish an order such as $XL > L > M$.
- On the other hand, nominal features do not possess any inherent order.
 - To extend the previous example, we can view T-shirt color as a nominal feature since it doesn't typically make sense to compare colors in terms of magnitude, such as stating that red is larger than blue.

Data Preprocessing

Handling categorical data

- To grasp the concepts better, let's generate a sample datasets using the **pandas** library.

```
@author: Mehdi
"""

import pandas as pd
df = pd.DataFrame([
    ['green', 'M', 10.1, 'class1'],
    ['red', 'L', 13.5, 'class2'],
    ['blue', 'XL', 15.3, 'class1']])
df.columns = ['color', 'size', 'price', 'classlabel']

print(df)
```

	color	size	price	classlabel
0	green	M	10.1	class1
1	red	L	13.5	class2
2	blue	XL	15.3	class1

- The DataFrame we created consists of various types of features.
 - It includes a **nominal feature (color)**, an **ordinal feature (size)**, and a **numerical feature (price)**. The class labels, assuming we generated the dataset for a supervised learning task, are located in the last column.

Handling categorical data: Mapping ordinal features

- To ensure proper interpretation of ordinal features by the learning algorithm, it is necessary to **convert categorical string values into integers**. However, there is no built-in function that can automatically determine the correct order of the labels in the size feature. **Therefore, we must define the mapping manually to establish the correct order.**
- we use the **map()** function to replace the ordinal labels in the 'size' column with their corresponding numerical values.

```
import pandas as pd

df = pd.DataFrame([
    ['green', 'M', 10.1, 'class1'],
    ['red', 'L', 13.5, 'class2'],
    ['blue', 'XL', 15.3, 'class1']])
df.columns = ['color', 'size', 'price', 'classlabel']

#Before Apply the mapping
print(df)

# Manually define the mapping for ordinal values
size_mapping = {'M': 1, 'L': 2, 'XL': 3}

# Apply the mapping to the 'size' column
print("\nApply the mapping to the 'size' column:")
df['size'] = df['size'].map(size_mapping)

print("\n",df)
```

	color	size	price	classlabel
0	green	M	10.1	class1
1	red	L	13.5	class2
2	blue	XL	15.3	class1

Apply the mapping to the 'size' column:

	color	size	price	classlabel
0	green	1	10.1	class1
1	red	2	13.5	class2
2	blue	3	15.3	class1

Handling categorical data: Encoding class labels

- In many machine learning libraries, it is necessary to encode class labels as integer values.
- While some classification estimators in scikit-learn automatically handle this conversion internally, it is still recommended to provide class labels as integer arrays to avoid potential issues.
- To encode the class labels, we can use a similar approach to the mapping of ordinal features discussed earlier.
- It's important to note that class labels are not ordinal, meaning the specific integer assigned to a string label is arbitrary.

Handling categorical data: Encoding class labels

- Example: To encode the class labels, we can use the `enumerate()` function to assign integer values starting from 0:
- In this code, we first create a mapping dictionary `class_mapping` using `enumerate()` and the unique class labels in the 'classlabel' column. Then, we use `map()` to replace the class labels with their corresponding integer values based on the mapping. As a result, the 'classlabel' column is encoded with integer values (0 and 1 in this case) for further use in machine learning algorithms.

```
import pandas as pd

df = pd.DataFrame([
    ['green', 'M', 10.1, 'class1'],
    ['red', 'L', 13.5, 'class2'],
    ['blue', 'XL', 15.3, 'class1']],
    columns=['color', 'size', 'price', 'classlabel'])

# Create a dictionary to map class labels to integers
class_mapping = {}
count = 0

for label in df['classlabel'].unique():
    class_mapping[label] = count
    count += 1

# Use the dictionary to encode class labels
df['classlabel'] = df['classlabel'].map(class_mapping)

print(df)
```

	color	size	price	classlabel
0	green	1	10.1	0
1	red	2	13.5	1
2	blue	3	15.3	0

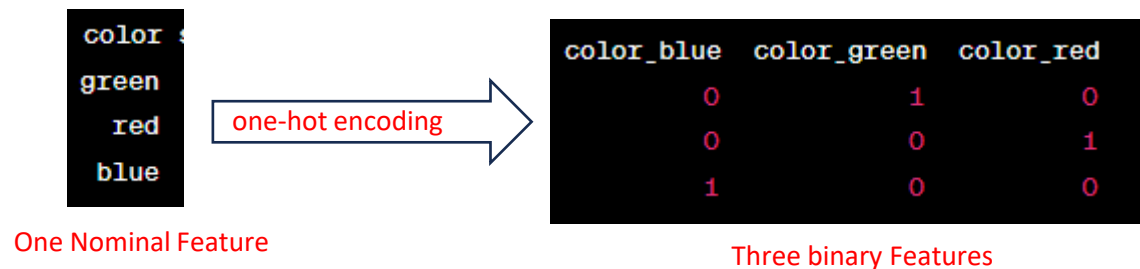
Handling categorical data: one-hot encoding on nominal features

- **One-hot encoding** is a technique used to represent categorical variables as binary vectors.
- **It is used when we have categorical features that are not ordinal**, and we want to represent each category as a separate binary feature. This is done to avoid any ordinal relationship assumptions between the categories.
- As an illustration, let's consider the encoding of colors: blue, yellow, and green are represented by 1, 2, and 3, respectively (or alternatively, as (0, 1, 2)). If an intelligent algorithm were to analyze this encoding, it might mistakenly infer that **$1 < 2 < 3$** , leading to the conclusion that **blue > yellow** and **yellow > green**.
- However, it's important to note that there is no inherent greater or smaller relationship between different colors based on this encoding.

Data Preprocessing

Handling categorical data: one-hot encoding on nominal features

- **one-hot encoding** creates binary features for each unique category in a categorical variable.
- Instead of assigning integer values, it represents each category as a separate binary feature (0 or 1).
- This encoding ensures that there is no ordinal relationship between the categories and prevents any misinterpretation of numerical values by the learning algorithms.



Data Preprocessing

Handling categorical data: one-hot encoding on nominal features

- Example:

```
@author: Mehdi
"""

import pandas as pd
from sklearn.preprocessing import OneHotEncoder

df = pd.DataFrame([
    ['green', 'M', 10.1, 'class1'],
    ['red', 'L', 13.5, 'class2'],
    ['blue', 'XL', 15.3, 'class1']],
    columns=['color', 'size', 'price', 'classlabel'])

# Extract the 'color' column as a separate DataFrame
color_df = df[['color']]

# Create an instance of OneHotEncoder
encoder = OneHotEncoder(sparse_output=False, dtype=int)

# Perform one-hot encoding on the 'color' column
color_encoded = encoder.fit_transform(color_df)

# Create a DataFrame with the encoded columns
color_encoded_df = pd.DataFrame(color_encoded, columns=[f'color_{category}' for category in encoder.categories_[0]])

# Concatenate the original DataFrame with the encoded columns
df_encoded = pd.concat([df.drop('color', axis=1), color_encoded_df], axis=1)

print(df_encoded)
```

	size	price	classlabel	color_blue	color_green	color_red
0	M	10.1	class1	0	1	0
1	L	13.5	class2	0	0	1
2	XL	15.3	class1	1	0	0

Class imbalance

- Class imbalance refers to a situation in machine learning and AI where the distribution of classes or categories in a dataset is not equal or is significantly skewed.
- In other words, one class may have a much larger number of instances or examples compared to the other class(es), resulting in an imbalanced representation.
- For example, let's consider a binary classification problem where the goal is to predict whether an email is spam or not spam. If the dataset contains 95% non-spam emails and only 5% spam emails, then we have a class imbalance. In this case, the majority class is non-spam, while the minority class is spam.

Class imbalance challenges

- Class imbalance can pose challenges for machine learning algorithms because they tend to be biased towards the majority class.
- They can achieve high accuracy by simply predicting the majority class for most instances, but this approach fails to effectively capture the patterns and characteristics of the minority class.
 - **Biased model performance:** Class imbalance can lead to biased models that perform poorly in predicting the minority class. Accuracy, which is a common evaluation metric, can be misleading in such cases, as a model that predicts only the majority class can still achieve high accuracy due to the imbalance.
 - **Reduced recall:** The recall, also known as sensitivity or true positive rate, measures the ability of a model to correctly identify positive instances (e.g., the minority class in an imbalanced dataset). Imbalanced datasets often result in lower recall for the minority class.
 - **Increased false positives:** Imbalanced datasets can cause models to produce a high number of false positives for the minority class. In our email example, this means incorrectly classifying non-spam emails as spam.

Class imbalance challenges

- Addressing class imbalance in machine learning is a specialized skill that requires careful consideration. Typically, there are two main categories of approaches to tackle this issue:
 - Undersampling is a technique used to address class imbalance by reducing the number of samples from the overrepresented class(es). It involves randomly selecting a fraction of samples or implementing more sophisticated strategies to remove specific data points. One approach is to prioritize the removal of samples that are similar to others in the dataset, such as using k-means clustering on the majority class or removing Tomek's links. However, it's important to be cautious of the potential side effects and implications of undersampling, as it can lead to information loss and a significant decrease in prediction accuracies for the majority classes if excessively applied.

Class imbalance challenges

- Addressing class imbalance in machine learning is a specialized skill that requires careful consideration. Typically, there are two main categories of approaches to tackle this issue:
 - **Oversampling** is a technique used to address class imbalance by generating synthetic data points for the minority classes. Unlike undersampling, oversampling aims to decrease the disparity in class sizes. However, simply generating artificial data and assigning it to the minority class can introduce unintended class characteristics that may mislead the learning process.
 - To avoid these pitfalls, specialized algorithms such as **SMOTE** (Synthetic Minority Over-sampling Technique) and **ADASYN** (Adaptive Synthetic Sampling) have been developed. These algorithms intelligently generate synthetic data in a manner that preserves the original characteristics of the minority class, preventing contamination of the dataset.
 - By employing oversampling techniques like SMOTE and ADASYN, it is possible to effectively address class imbalance without distorting the underlying distribution and compromising the quality of the training data.

Class imbalance: address challenges

- To mitigate the negative effects of oversampling and undersampling, it is possible to combine these techniques in a strategic manner. One common approach involves first oversampling the minority class to increase its representation, and then undersampling to reduce the discrepancy in class distribution.
- Class imbalance is a prevalent issue in machine learning, and there are dedicated software libraries designed to tackle it. An example is **Imbalanced-learn**, which provides a diverse set of data resampling techniques tailored for different problems. Similar to classifiers, different resampling techniques exhibit unique characteristics and may be more suitable for specific problem scenarios.
- By leveraging these resampling techniques and software libraries, practitioners can effectively address class imbalance and improve the performance of their machine learning models.

Performance evaluation metrics

- There are several metrics commonly used to evaluate the performance of machine learning models:
 - Accuracy, Precision, Recall, F1 Score, ...
- The choice of metric depends on the specific problem and the type of model being used.
- It's important to select appropriate metrics that align with the objectives of your project.

Performance evaluation metrics: Confusion Matrix

- A confusion matrix is a tabular representation that shows the performance of a classification model by comparing the predicted labels with the actual labels of a dataset. It provides a comprehensive overview of the model's performance by breaking down the predictions into four categories:
 - **True Positives (TP)**: These are the instances that are correctly predicted as positive by the model.
 - **True Negatives (TN)**: These are the instances that are correctly predicted as negative by the model.
 - **False Positives (FP)**: These are the instances that are incorrectly predicted as positive by the model when the actual label is negative.
 - **False Negatives (FN)**: These are the instances that are incorrectly predicted as negative by the model when the actual label is positive.

	Predicted Negative	Predicted Positive
Actual Negative	TN	FP
Actual Positive	FN	TP

Performance evaluation metrics: Confusion Matrix

- Example: Let's say we have a binary classification problem where we are predicting whether an email is spam or not. The confusion matrix would look like this:

	Predicted Negative	Predicted Positive
Actual Negative	900	10
Actual Positive	20	70

- In this example, we have:
 - True Negatives (TN): 900 emails that were actually not spam and were correctly predicted as not spam.
 - False Positives (FP): 10 emails that were actually not spam but were incorrectly predicted as spam.
 - False Negatives (FN): 20 emails that were actually spam but were incorrectly predicted as not spam.
 - True Positives (TP): 70 emails that were actually spam and were correctly predicted as spam.

Performance evaluation metrics: Confusion Matrix

- The confusion matrix allows us to calculate various performance metrics such as accuracy, precision, recall, and F1 score. These metrics can provide insights into the model's performance, particularly in terms of its ability to correctly classify instances from different classes.
- By analyzing the values in the confusion matrix, we can understand how well the model is performing and identify any potential biases or errors it may have. It helps in evaluating the strengths and weaknesses of the classification model and aids in making informed decisions about the model's suitability for a particular task.

Performance evaluation metrics: Accuracy

- **Accuracy** is a performance metric that measures the overall correctness of a classification model.
 - It represents the proportion of correctly classified instances (both true positives and true negatives) out of the total number of instances in the dataset.

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

- Accuracy provides a useful overall measure of how well the model is performing, especially in balanced datasets where the classes are relatively equal in size.
- However, accuracy can be misleading when dealing with imbalanced datasets, where the number of instances in different classes is significantly different. In such cases, it's important to consider additional evaluation metrics such as precision, recall, and F1 score to get a more comprehensive understanding of the model's performance.

Performance evaluation metrics: Accuracy

- **Example:** Let's consider a binary classification problem where we are predicting whether an email is spam (positive class) or not spam (negative class). Here's an example confusion matrix:

	Predicted Negative	Predicted Positive
Actual Negative	TN = 900	FP = 10
Actual Positive	FN = 20	TP = 70

- To calculate accuracy using the confusion matrix, we can use the formula:
 - $\text{Accuracy} = (TN + TP) / (TN + FP + FN + TP)$
 - Plugging in the values from the confusion matrix:
 - $\text{Accuracy} = (900 + 70) / (900 + 10 + 20 + 70) = 970 / 1000 = 0.97$ (or 97%)
 - So, in this example, the accuracy of the classification model is 97%, indicating that it correctly classified 97% of the emails in the dataset.

Performance evaluation metrics: Precision

- **Precision** is a performance metric that measures the proportion of correctly predicted positive instances (true positives) out of the total instances predicted as positive (true positives and false positives).
 - It quantifies the model's ability to correctly identify positive instances without falsely classifying too many negative instances as positive.

$$\text{Precision} = \frac{TP}{TP + FP}$$

- Precision focuses on the accuracy of positive predictions and is particularly useful when the cost of false positives is high.
- A high precision value indicates a low false positive rate, meaning that the model is making fewer incorrect positive predictions.
- Precision is often used in combination with other evaluation metrics such as recall and F1 score to provide a comprehensive understanding of the model's performance.

Performance evaluation metrics: Precision

- **Example:** Let's consider a binary classification problem where we are predicting whether an email is spam (positive class) or not spam (negative class). Here's an example confusion matrix:

	Predicted Negative	Predicted Positive
Actual Negative	TN = 900	FP = 10
Actual Positive	FN = 20	TP = 70

- To calculate precision using the confusion matrix, we can use the formula:
 - $\text{Precision} = \text{TP} / (\text{TP} + \text{FP})$
 - Plugging in the values from the confusion matrix:
 - $\text{Precision} = 70 / (70 + 10) = 70 / 80 = 0.875$ (or 87.5%)
 - In this example, the precision of the classification model is 87.5%. This means that out of all the instances predicted as positive, 87.5% were actually correctly classified as positive (spam). It indicates that the model has a relatively low false positive rate, making fewer incorrect positive predictions.

Performance evaluation metrics: Recall

- Recall, also known as sensitivity or true positive rate, is a performance metric that measures the proportion of correctly predicted positive instances (true positives) out of the total actual positive instances.
 - It quantifies the model's ability to identify all positive instances correctly, without missing any.

$$Recall = \frac{TP}{TP + FN}$$

- Recall focuses on the ability to capture positive instances and is particularly important when the cost of false negatives is high.
- A high recall value indicates a low false negative rate, meaning that the model is correctly identifying a larger proportion of positive instances.
- Recall is often used in combination with other evaluation metrics such as precision and F1 score to provide a comprehensive understanding of the model's performance.

Performance evaluation metrics: Recall

- **Example:** Let's consider a binary classification problem where we are predicting whether an email is spam (positive class) or not spam (negative class). Here's an example confusion matrix:

	Predicted Negative	Predicted Positive
Actual Negative	TN = 900	FP = 10
Actual Positive	FN = 20	TP = 70

- To calculate Recall using the confusion matrix, we can use the formula:
 - $\text{Recall} = \text{TP} / (\text{TP} + \text{FN})$
 - Plugging in the values from the confusion matrix:
 - $\text{Recall} = 70 / (70 + 20) = 70 / 90 = 0.7778$ (or 77.78%)
 - In this example, the recall of the classification model is 77.78%. This means that out of all the actual positive instances (spam emails), the model correctly identified 77.78% of them as positive. It indicates that the model has a relatively low false negative rate, as it captures a significant proportion of positive instances.

Performance evaluation metrics: F1-Score

- The **F1 score** is a metric that combines both precision and recall into a single value, providing a balanced measure of a classification model's performance.
 - It is the harmonic mean of precision and recall, and it takes into account both false positives and false negatives.

$$F_1\text{-score} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} = \frac{2TP}{2TP + FP + FN}$$

- The F1 score ranges from 0 to 1, where 1 indicates the best performance and 0 indicates the worst. It seeks to find a balance between precision and recall, giving equal importance to both metrics.
- The F1 score is particularly useful when the dataset is imbalanced or when false positives and false negatives have different costs. It provides a single value that summarizes the model's ability to balance precision and recall, taking into account both the true positives and false negatives. By considering both precision and recall, the F1 score provides a more comprehensive evaluation of the model's overall performance compared to using precision or recall alone.

Performance evaluation metrics: F1-Score

- **Example:** Let's consider a binary classification problem where we are predicting whether an email is spam (positive class) or not spam (negative class). Here's an example confusion matrix:

	Predicted Negative	Predicted Positive
Actual Negative	TN = 900	FP = 10
Actual Positive	FN = 20	TP = 70

- To calculate F1 Score using the confusion matrix, we can first calculate precision and recall:
 - **Precision** = $TP / (TP + FP) = 70 / (70 + 10) = 0.875$
 - **Recall** = $TP / (TP + FN) = 70 / (70 + 20) = 0.7778$
- Now, we can calculate the F1 score using the formula:
 - **F1 score** = $2 * (Precision * Recall) / (Precision + Recall) = 2 * (0.875 * 0.7778) / (0.875 + 0.7778) = 0.8235$
 - In this example, the F1 score of the classification model is 0.8235. This indicates that the model has achieved a balanced performance in terms of precision and recall, taking into account both false positives and false negatives. It is a measure of the model's overall effectiveness in classifying spam emails, considering both the correct identification of positive instances and the avoidance of false positives and false negatives.

Performance evaluation metrics: Receiver Operating Characteristic (ROC)

- ROC Curve:
 - The ROC curve is a graphical representation of the performance of a binary classification model at different classification thresholds.
 - It plots the True Positive Rate (TPR) against the False Positive Rate (FPR) at various threshold settings.
 - The TPR (also called sensitivity or recall) represents the proportion of actual positive instances correctly classified as positive. $TPR = TP / (TP + FN)$
 - The FPR represents the proportion of actual negative instances incorrectly classified as positive. $FPR = FP / (FP + TN)$
 - The ROC curve illustrates the trade-off between TPR and FPR as the classification threshold changes.
 - The ideal ROC curve hugs the top-left corner of the plot, indicating high TPR and low FPR across all threshold values.

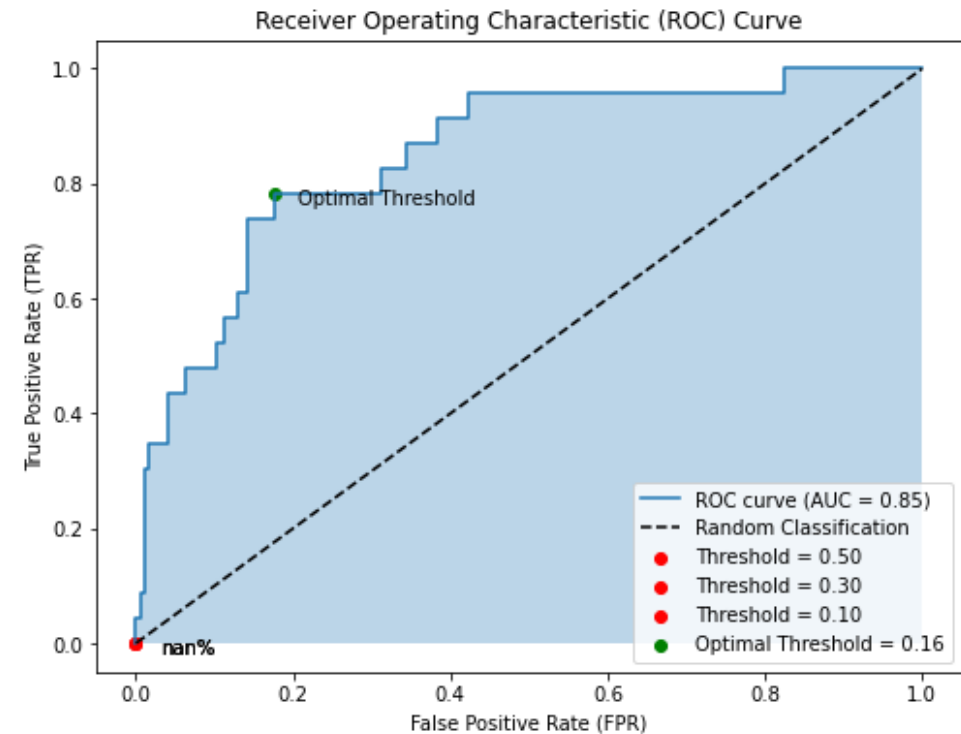
Performance evaluation metrics: Area Under Curve (AUC)

- AUC (Area Under the Curve):
 - The **AUC** is a scalar value that quantifies the overall performance of a binary classification model based on the ROC curve.
 - It represents the area under the ROC curve, which **ranges between 0 and 1**.
 - **A perfect classifier** has an **AUC of 1**, indicating that it achieves a TPR of 1 while maintaining an FPR of 0 across all thresholds.
 - **A random classifier** that performs no better than chance has **an AUC of 0.5**, resulting in a diagonal line in the ROC plot.
 - **A higher AUC value indicates better model performance** and a stronger ability to distinguish between positive and negative instances.
- In summary, the ROC curve visually depicts the trade-off between the true positive rate and false positive rate at different classification thresholds. The AUC provides a single metric to evaluate the model's overall performance, where a higher value indicates better classification ability.

Performance evaluation metrics: ROC and AUC

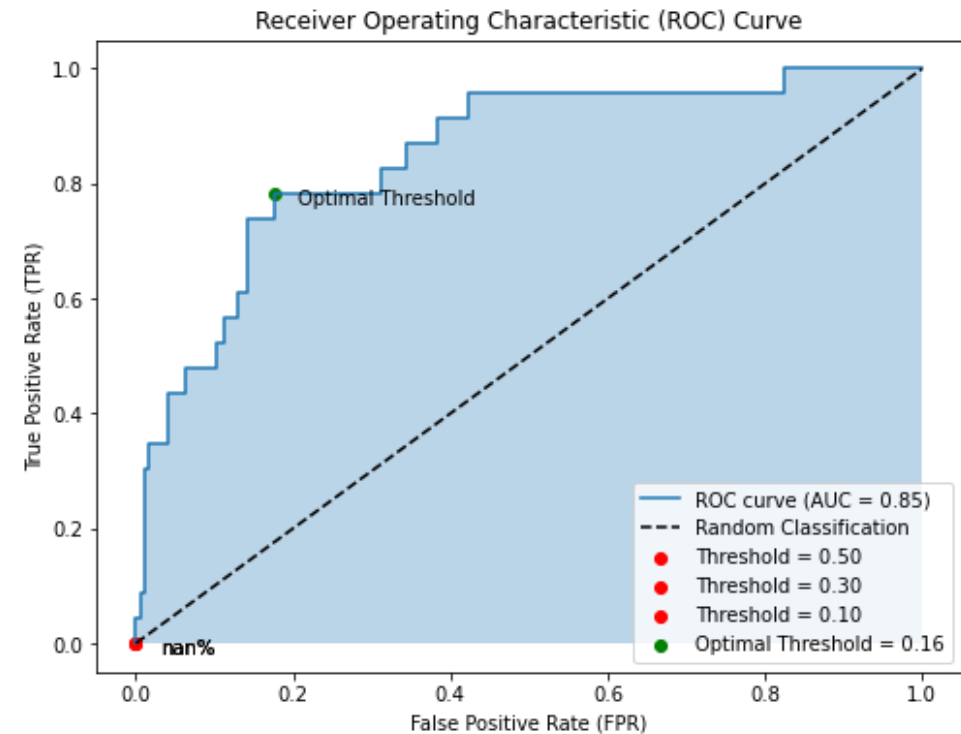
- Example:

- The plot represents the Receiver Operating Characteristic (ROC) curve, which is a graphical representation of the performance of a binary classification model. Here's a brief explanation of the plot:
- The blue line represents the ROC curve. It shows the relationship between the True Positive Rate (TPR) and the False Positive Rate (FPR) at different classification thresholds.
- The black dashed line is the baseline or random classification line. It represents the expected performance of a random classifier.
- The red dots on the ROC curve represent specific threshold values. Each dot indicates the TPR and FPR achieved when the classification threshold is set to that value.



Performance evaluation metrics: ROC and AUC

- Example:
 - The green dot represents the optimal threshold, which is determined based on the Youden's J statistic ($J = \text{sensitivity} + \text{specificity} - 1$). The optimal threshold is the point on the ROC curve that maximizes the difference between TPR and FPR, indicating the best trade-off between sensitivity and specificity.
 - The shaded area under the ROC curve represents the Area Under the Curve (AUC), which is a measure of the overall performance of the classifier. A higher AUC value indicates better classification performance.
 - The plot helps in visualizing how the classifier performs at different threshold levels. It allows you to assess the balance between TPR and FPR and make informed decisions about the threshold selection based on the specific requirements of the problem.



Scaling

- **Scaling** is the process of transforming the values of numerical features in a dataset to a specific range or distribution.
- It is **a preprocessing step** that aims to bring all features to a similar scale, which can help improve the performance and stability of machine learning algorithms.
- The need for scaling arises when the features in a dataset have different scales or units.
- If the features have significantly different ranges, it can lead to biased results and inaccurate model performance.
- Scaling the features ensures that they are all on a similar scale, allowing for fair comparison and avoiding dominance by features with larger values.

Scaling

- **Min-Max Scaling (Normalization):** It scales the values of features to a specific range, typically between 0 and 1. It subtracts the minimum value from each value and then divides by the range (maximum value minus the minimum value). This method preserves the original distribution of the data while ensuring it falls within the desired range.
- **Standardization (Z-score Scaling):** It transforms the values of features to have a mean of 0 and a standard deviation of 1. It subtracts the mean value from each value and then divides by the standard deviation. Standardization assumes that the data follows a Gaussian distribution, and it centers the data around 0, allowing for meaningful comparisons of the features.

Scaling

- Scaling is important in various scenarios:
 - **Distance-Based Algorithms:** Scaling is crucial for algorithms that rely on distance calculations, such as K-nearest neighbors (KNN) and clustering algorithms. Since these algorithms use distances between data points, scaling ensures that each feature contributes proportionally to the distance calculations.
 - **Regularization Techniques:** Scaling is often used in regularization techniques, such as Ridge and Lasso regression. These techniques penalize large coefficients, and scaling helps to avoid attributes with larger scales dominating the regularization process.
 - **Neural Networks:** Scaling is commonly applied to features in neural networks to ensure stable and efficient convergence during training. Scaling helps in preventing large weight updates and avoids issues such as vanishing or exploding gradients.
 - **Optimization Algorithms:** Scaling can improve the convergence of optimization algorithms, such as gradient descent. When features have different scales, the optimization process can be slowed down or become unstable. Scaling the features helps the algorithm converge more efficiently.

Data Preprocessing

Scaling

- Example: Let's consider a numeric example for both standardization and normalization using a sample dataset of temperatures (in Celsius):
 - Temperatures = [25, 30, 28, 20, 22, 27, 26]
 - Standardization: To standardize the temperatures, we need to calculate the mean and standard deviation first.
 - Mean = $(25 + 30 + 28 + 20 + 22 + 27 + 26) / 7 = 24.57$ Standard Deviation = $\sqrt{((25 - 24.57)^2 + (30 - 24.57)^2 + (28 - 24.57)^2 + (20 - 24.57)^2 + (22 - 24.57)^2 + (27 - 24.57)^2 + (26 - 24.57)^2) / 7} = 3.03$
 - Now, we can calculate the standardized values using the formula:
 - Standardized Value = $(x - \text{mean}) / \text{standard deviation}$
 - For each temperature value, we can compute the standardized value:
 - Standardized Temperatures = $[(25 - 24.57) / 3.03, (30 - 24.57) / 3.03, (28 - 24.57) / 3.03, (20 - 24.57) / 3.03, (22 - 24.57) / 3.03, (27 - 24.57) / 3.03, (26 - 24.57) / 3.03]$
 - Standardized Temperatures = [0.14, 1.45, 0.94, -1.44, -1.03, 0.63, 0.32]

Data Preprocessing

Scaling

- Example

- **Normalization:** To normalize the temperatures, we need to find the minimum and maximum values first.
 - Minimum Temperature = 20 Maximum Temperature = 30
 - Now, we can calculate the normalized values using the formula:
 - **Normalized Value = $(x - \text{min}) / (\text{max} - \text{min})$**
 - For each temperature value, we can compute the normalized value:
 - Normalized Temperatures = $[(25 - 20) / (30 - 20), (30 - 20) / (30 - 20), (28 - 20) / (30 - 20), (20 - 20) / (30 - 20), (22 - 20) / (30 - 20), (27 - 20) / (30 - 20), (26 - 20) / (30 - 20)]$
 - Normalized Temperatures = **$[0.50, 1.00, 0.80, 0.00, 0.20, 0.70, 0.60]$**
- In both standardization and normalization, the resulting values are scaled to a specific range. In standardization, the values are transformed to have a mean of 0 and a standard deviation of 1, while in normalization, the values are transformed to fall between 0 and 1.
- Please note that these are simplified examples, and in practice, libraries such as NumPy or scikit-learn provide convenient functions for standardization and normalization that can be used in your code.

Data Preprocessing

Scaling

- Example using Numpy:

```
@author: Mehdi
"""

import numpy as np

# Temperature data
temperatures = np.array([25, 30, 28, 20, 22, 27, 26])

# Standardization
mean = np.mean(temperatures)
std = np.std(temperatures)
standardized_temperatures = (temperatures - mean) / std

print("\n\nStandardized Temperatures:", standardized_temperatures)

# Normalization
min_temp = np.min(temperatures)
max_temp = np.max(temperatures)
normalized_temperatures = (temperatures - min_temp) / (max_temp - min_temp)

print("\n\nNormalized Temperatures:", normalized_temperatures)
```

```
Standardized Temperatures: [-0.13389655  1.4282299   0.80337932 -1.696023
-1.07117242  0.49095403
 0.17852874]
```

```
Normalized Temperatures: [0.5 1.  0.8 0.  0.2 0.7 0.6]
```

Data Preprocessing

Scaling

- Example using sklearn:

```
@author: Mehdi
"""

import numpy as np
from sklearn.preprocessing import StandardScaler, MinMaxScaler

# Temperature data
temperatures = np.array([[25], [30], [28], [20], [22], [27], [26]])

# Normalization
min_max_scaler = MinMaxScaler()
normalized_temperatures = min_max_scaler.fit_transform(temperatures)

print("Normalized Data:")
print(normalized_temperatures)

# Standardization
standard_scaler = StandardScaler()
standardized_temperatures = standard_scaler.fit_transform(temperatures)

print("Standardized Data:")
print(standardized_temperatures)
```

```
Normalized Data:
[[0.5]
 [1. ]
 [0.8]
 [0. ]
 [0.2]
 [0.7]
 [0.6]]
Standardized Data:
[[-0.13389655]
 [ 1.4282299 ]
 [ 0.80337932]
 [-1.696023  ]
 [-1.07117242]
 [ 0.49095403]
 [ 0.17852874]]
```

Data Preprocessing

What we covered

- Data Preprocessing ☒
- Missing data handling ☒
- Categorical data handling ☒
- Class Imbalance ☒
- Performance evaluation metrics ☒
- Feature Scaling ☒

References

- Raschka, S., 2015. Python machine learning: unlock deeper insights into machine learning with this vital guide to cutting-edge predictive analytics. *(No Title)*.
- Chio, C. and Freeman, D., 2018. *Machine learning and security: Protecting systems with data and algorithms*. " O'Reilly Media, Inc."